

PART A

Experiment No.02

A.1 Aim: To Implement uninformed search methods using C,python or Java.

A.2 Prerequisite: Data Structure, Searching Techniques

A.3 Outcome:

After successful completion of this experiment students will be able to

- Solve the problems like water jug, tic-tac-toe using uninformed search methods like DFS/BFS.

Tools Required: C /Java

A.4 Theory:

A problem determines the graph and the goal but not which path to select from the frontier. This is the job of a search strategy. A search strategy specifies which paths are selected from the frontier. Different strategies are obtained by modifying how the selection of paths in the frontier is implemented.

The **uninformed search strategies** that do not take into account the location of the goal. Intuitively, these algorithms ignore where they are going until they find a goal and report success.

1. Depth-First Search
2. Breadth-First Search
3. Lowest-Cost-First Search

1. **Depth-First Search:** The first strategy is **depth-first search**. In depth-first search, the frontier acts like a last-in first-out **stack**. The elements are added to the stack one at a time. The one selected and taken off the frontier at any time is the last element that was added.

Algorithm:

DEPTH(AdMax): Depth first search in state space

1. Init lists $OPEN \leftarrow \{S_i\}$, $CLOSED \leftarrow \{\}$ 2.

if $OPEN = \{\}$

then return FAIL

3. Remove first node S from OPEN and insert it into CLOSED

3'. **if** $Ad(S) = AdMax$ **then repeat from 2**

4. Expand node S

4.1. Generate all successors S_j of node S

4.2. **for** each successor S_j of S **do**

4.2.1. Set link $S_j \rightarrow S$ 4.2.2.

if S_j is final state

then

i. Solution is $(S_j, \dots, S_i)$

ii. **return** SUCCESS

4.2.3. Insert S_j in OPEN, **at the beginning**

5. **repeat from 2 end.**

Example:



Algorithm BREADTH: **Breadth first search in state space**

if OPEN = {}

3. Remove first node S from OPEN and insert it in CLOSED

4.1. Generate all direct successors S_i of node S

4.2.1. Make link $S_j \rightarrow S$

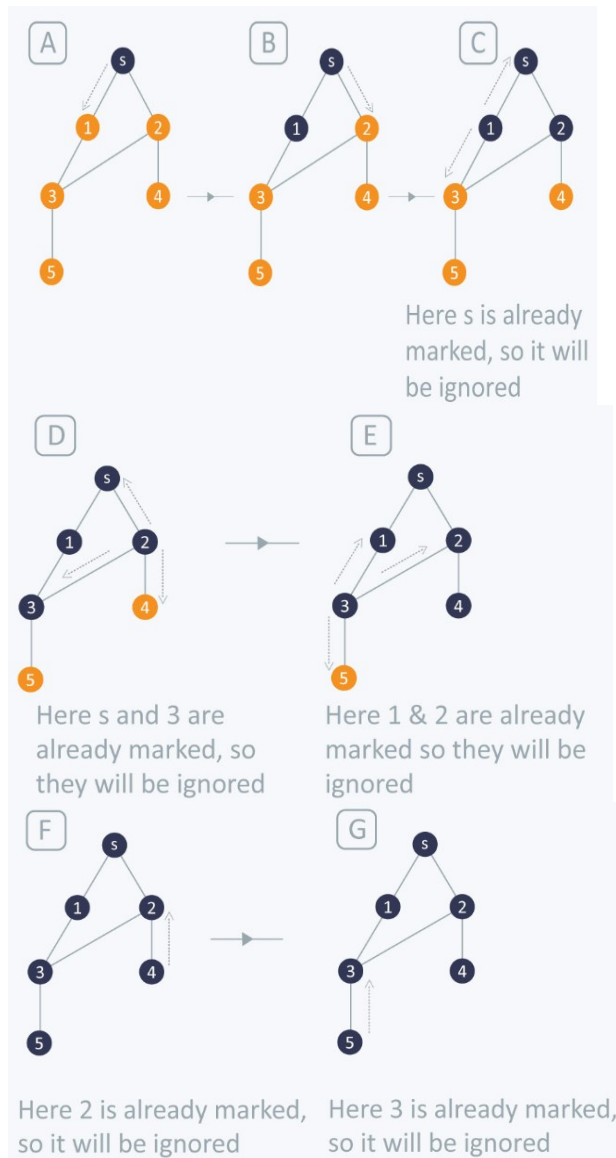
4.2.2. if S_j is final state

i. Solution is $(S_j, S, \dots, S_i)$

ii. **return SUCCESS**

4.2.3. Insert S_j in OPEN, at the end 5. **repeat from 2 end.**

Example:



PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.: C-54	Name: Quazi Md Moinuddin
Class: TE Comps C	Batch:C3
Date of Experiment: 14.01.25	Date of Submission:5.02.25
Grade:	

B.1 Software Code written by student: BFS:

```
graph = {
    'A': ['B', 'C', 'D'],
    'B': ['A', 'E'],
    'C': ['A', 'D'],
    'D': ['A', 'C', 'E'],
    'E': ['B', 'D', 'F'],
    'F': ['E', 'G'],
    'G': ['F']
}

# to print a BFS of a graph def
bfs(node):
    # mark vertices as False means not visited
    visited = []
    # make an empty queue for BFS
    queue = []
    # mark given node as visited and add it to the queue
    visited.append(node)    queue.append(node)

    while queue:
        # Remove the front vertex or the vertex at the 0th index from the queue
        v = queue.pop(0)
```

```
print(v, end=" ")

# Get all adjacent nodes of the removed node v from the graph hash table
# If an adjacent node has not been visited yet,
# then mark it as visited and add it to the queue
for neigh in graph[v]:      if neigh not in visited:
visited.append(neigh)
queue.append(neigh)

# Driver Code if __name__
== "__main__":
    bfs('A')
```

DFS: graph

```
= {
    'A': ['B', 'C', 'D'],
    'B': ['A', 'E'],
    'C': ['A', 'D'],
    'D': ['A', 'C', 'E'],
    'E': ['B', 'D', 'F'],
    'F': ['E', 'G'],
    'G': ['F']
}
```

```
# DFS function def dfs(node,
visited=None):    # Initialize visited
set if not provided    if visited is None:
visited = set()
    # Print current node if not visited
if node not in visited:
print(node, end=" ")
visited.add(node)
```

```
    # Recursive call for all adjacent nodes    for neighbor in
graph[node]:        if neighbor not in visited: # Make sure we visit a
node only once        dfs(neighbor, visited)

# Driver code if __name__
== "__main__":
    dfs('A')
```

B.2 Input and Output:

BFS:

```
graph = {
    'A': ['B', 'C', 'D'],
    'B': ['A', 'E'],
    'C': ['A', 'D'],
    'D': ['A', 'C', 'E'],
    'E': ['B', 'D', 'F'],
    'F': ['E', 'G'],
    'G': ['F']
}

# to print a BFS of a graph
def bfs(node):
    # mark vertices as False means not visited
    visited = []
    # make an empty queue for BFS
    queue = []
    # mark given node as visited and add it to the queue
    visited.append(node)
    queue.append(node)
    while queue:
        # Remove the front vertex or the vertex at the 0th index from the queue
        v = queue.pop(0)
        print(v, end=" ")
        # Get all adjacent nodes of the removed node v from the graph hash table
        # If an adjacent node has not been visited yet,
        # then mark it as visited and add it to the queue
        for neigh in graph[v]:
            if neigh not in visited:
                visited.append(neigh)
                queue.append(neigh)

# Driver Code
if __name__ == "__main__":
    bfs('A')
```

A B C D E F G

DFS:

```
graph = {
    'A': ['B', 'C', 'D'],
    'B': ['A', 'E'],
    'C': ['A', 'D'],
    'D': ['A', 'C', 'E'],
    'E': ['B', 'D', 'F'],
    'F': ['E', 'G'],
    'G': ['F']
}

# DFS function
def dfs(node, visited=None):
    # Initialize visited set if not provided
    if visited is None:
        visited = set()
    # Print current node if not visited
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
    # Recursive call for all adjacent nodes
    for neighbor in graph[node]:
        if neighbor not in visited: # Make sure we visit a node only once
            dfs(neighbor, visited)

# Driver code
if __name__ == "__main__":
    dfs('A')
```

A B E D C F G

B.3 Observations and learning:

We learnt about uninformed search algorithms in detail. We learnt many different uninformed algorithms. Here we focused on two algorithms in particular Breadth First Search and Depth First Search.

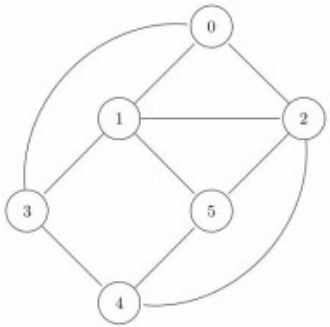
B.4 Conclusion:

The implementation of uninformed search methods such as Breadth-First Search (BFS), Depth-First Search (DFS) in languages like C, Python, or Java showcases their practical applications and differences.

- **BFS** explores all nodes level by level, guaranteeing the shortest path in an unweighted graph but can be memory-intensive, especially for large search spaces. It is useful when we need to find the shortest path or ensure all nodes at a certain depth are processed first.
- **DFS** follows a deep exploration strategy, going as deep as possible before backtracking. It is more memory-efficient than BFS and can be useful in scenarios where we need to explore all possibilities, but it may not find the shortest path and can get stuck in infinite loops if not properly handled.

B.5 Question of Curiosity

Q1) Which sequence corresponds to that of depth first search for the graph given below. The search starts at vertex 0 and lexicographic ordering is assumed for the edges emanating from each vertex.



- A. 0 1 2 4 3 5
- B. 0 1 2 5 4 3
- C. 0 1 2 3 4 5
- D. 0 1 3 4 2 5

Ans: D

Q2) Given a rooted tree, one desires to find the shortest path from the root to a given node v . Which algorithm would one use to find this shortest path?

- A. DFS
- B. BFS
- C. Either BFS or DFS

Ans: B

Q3) Consider a graph G . Let T be a BFS tree with root r . Let $d(u,v)$ denote the length of the shortest path between the nodes u and v . If v is visited before u in the breadth first search traversal, which of the following statements is true ?

- A. $d(r,v) > d(r,u)$
- B. $d(r,v) = d(r,u)$
- C. $d(r,v) < d(r,u)$
- D. insufficient information to comment on $d(r,v)$ and $d(r,u)$

Ans: C